



Babel vs OXC: Understanding the Future of JavaScript Tooling



Devendra Dhote

Follow

3 min read · Jun 23, 2026



123

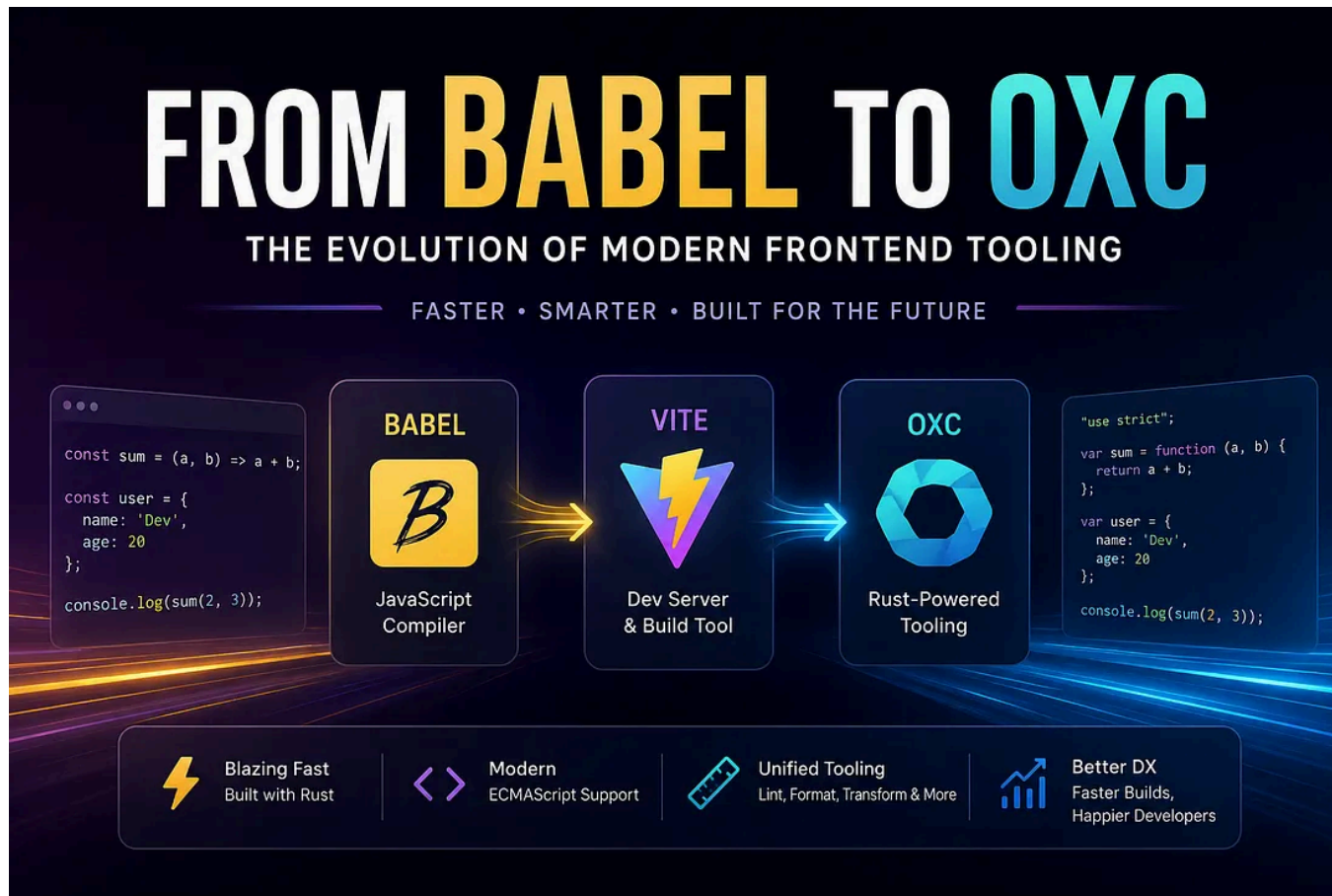


6



14





As frontend developers, we often focus on frameworks like React, Vue, or Next.js. However, the tools working behind the scenes have evolved just as rapidly.

Recently, I've been exploring how modern build tools are becoming significantly faster, and one trend stands out: the industry's gradual shift

from JavaScript-based tooling like Babel to Rust-powered tooling such as OXC.

Let's understand why.

. . .

What Is Babel?

Babel is a JavaScript compiler that converts modern JavaScript into code that older browsers can understand.

For example:

```
const sum = (a, b) => a + b;
```

Babel transforms it into something closer to:

```
var sum = function (a, b) {  
  return a + b;  
};
```

```
};
```

This allowed developers to use modern JavaScript features without worrying about browser compatibility.

Babel also played a huge role in React's success by transforming JSX into JavaScript.

```
<h1>Hello React</h1>
```

becomes:

```
React.createElement(  
  "h1",  
  null,  
  "Hello React"  
);
```

For many years, Babel became one of the most important tools in the frontend ecosystem.

. . .

The Challenge

As applications grew larger, build times also increased.

Every code change requires tooling to:

```
Parse
↓
Transform
↓
Bundle
↓
Reload
```

While Babel is incredibly powerful, it is written in JavaScript, which eventually becomes a limitation when processing very large codebases.

This led the ecosystem to search for faster alternatives.

. . .

The Rise of Native Tooling

Over the last few years, we've seen the emergence of tools such as:

- SWC (Speedy Web Compiler)
- Turbopack
- Rspack
- OXC (Oxidation Compiler)

Most of these tools are written in Rust.

Why Rust?

Because it offers:

- Better performance
- Lower memory usage
- Faster execution

- Excellent safety guarantees

This makes it ideal for build systems and compilers.

• • •

Where Does Vite Fit In?

Many developers think Vite is just a bundler, but that's not entirely accurate.

Get Devendra Dhote's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe



Remember me for faster sign in

Vite is primarily a build tool and development server.

One reason Vite became popular is its speed. Instead of relying heavily on traditional JavaScript tooling, it embraced faster compilation strategies and modern browser capabilities.

Now the ecosystem is taking another step forward with OXC.

. . .

What Is OXC?

OXC is a high-performance JavaScript and TypeScript tooling project written in Rust.

Its goal is to provide fast implementations of tools that developers use every day, including:

- Parsing
- Transforming
- Linting
- Formatting
- Minification

In simple terms, OXC aims to become a modern foundation for frontend tooling.

• • •

Babel vs OXC

Babel

Pros

- Mature ecosystem
- Extensive plugin support
- Battle-tested

Cons

- Written in JavaScript
- Slower on large projects

OXC

Pros

- Written in Rust

- Extremely fast
- Lower resource consumption

Cons

- Newer ecosystem
- Still evolving

• • •

Why Developers Should Care

A faster toolchain directly improves the developer experience.

Faster Startup

↓

Faster Rebuilds

↓

Faster Hot Reloading

↓

More Productive Development

Even saving a few hundred milliseconds on every rebuild adds up significantly over the lifetime of a project.

. . .

Final Thoughts

Babel helped developers embrace modern JavaScript.

Vite improved the frontend development experience.

OXC represents the next step in this evolution by bringing Rust-level performance to JavaScript tooling.

While Babel remains an essential part of the ecosystem, it's exciting to see how projects like OXC are pushing frontend tooling toward a faster and more efficient future.

As developers, understanding these tools helps us appreciate what happens behind the scenes every time our application starts instantly or reloads in milliseconds.